

정적분석으로 LLM 가이드 하여 파이썬 타입분석 잘하기

오원석, Michael Pradel, 오학주

풀고자 하는 문제

1. 파이썬 프로그램을 받음
2. 타입이 없는 함수를 골라 타입을 달음

Unannotated

```
import Path
def foo(p):
    return Path(p+".py")
```

⇒

Annotated

```
import Path
def foo(p: str) -> Path:
    return Path(p+".py")
```

3. 모든 함수에 타입이 달릴 때까지 반복

풀고자 하는 문제

1. 파이썬 프로그램을 받음
2. 타입이 없는 함수를 골라 타입을 달음

Unannotated

```
import Path
def foo(p):
    return Path(p+".py")
```

⇒

Annotated

```
import Path
def foo(p: str) -> Path:
    return Path(p+".py")
```

3. 모든 함수에 타입이 달릴 때까지 반복



풀고자 하는 문제

1. 파이썬 프로그램을 받음
2. 타입이 없는 함수를 골라 타입을 달음

Unannotated

```
import Path
def foo(p):
    return Path(p+".py")
```

⇒

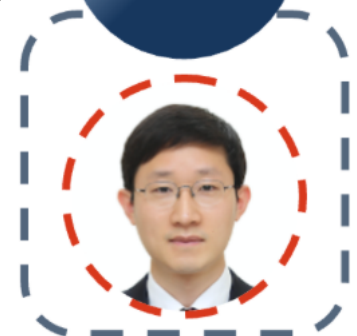
Annotated

```
import Path
def foo(p: str) -> Path:
    return Path(p+".py")
```

3. 모든 함수에 타입이 달릴 때까지 반복

파이썬에서 검증하기 위해서는
타입 정보가 필수

SW 패치검증
전문가



기존 연구

- ICLR'23: Seq2Seq 모델링하여 푸니까 잘 되더라
- ICSE'25: Ranking하는 모델 학습하니까 잘 되더라
- ICSE'26: 코드 패턴으로 Re-Ranking하니까 잘 되더라

기존 연구

- ICLR'23: Seq2Seq 모델링하여 푸니까 잘 되더라
- ICSE'25: Ranking하는 모델 학습하니까 잘 되더라
- ICSE'26: 코드 패턴으로 Re-Ranking하니까 잘 되더라

CodeT5(700M)에서 놀고 있다

기존 연구

- ICLR'23: Seq2Seq 모델링하여 푸니까 잘 되더라
- ICSE'25: Ranking하는 모델 학습하니까 잘 되더라
- ICSE'26: 코드 패턴으로 Re-Ranking하니까 잘 되더라

CodeT5(700M)에서 놀고 있다

대 LLM 시대를 전혀 반영하고 있지 않다

대 LLM 시대

- Codex, Claude Code, Aider 등 LLM 활용 도구들이 코딩 문제를 잘 풀기 시작함
- **장점)** 월등한 성능
 - 69.3% (CodeT5) vs 82.41% (gpt-oss:120b)

대 LLM 시대

- Codex, Claude Code, Aider 등 LLM 활용 도구들이 코딩 문제를 잘 풀기 시작함
- **장점)** 월등한 성능
 - 69.3% (CodeT5) vs 82.41% (gpt-oss:120b)
- **단점)** 전혀 효율적이지 않다
 - 고작 **14k 프로그램** 타입 다는데 무려 **\$57.67*** 소모

목표

- 어떻게든 LLM 호출을 최소화 할 수 있을까?
- 어떻게든 컨텍스트를 최소화 할 수 있을까?
 - 두 개를 해결하면 Cost 문제가 해결된다
- **기본 아이디어)** 정적분석을 활용하면 될 것 같다

기존 도구들의 문제점

못해

마음대로

대충

기존 도구들의 문제점

정적분석 제대로 활용 못해

마음대로

대충

기존 도구들의 문제점

정적분석 제대로 활용 못해

함수 호출순서 마음대로

대충

기존 도구들의 문제점

정적분석 제대로 활용 못해

함수 호출순서 마음대로

컨텍스트 생성 대충

기존 도구들의 문제점

정적분석 제대로 활용 못해

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```


기존 도구들의 문제점

정적분석 제대로 활용 못해

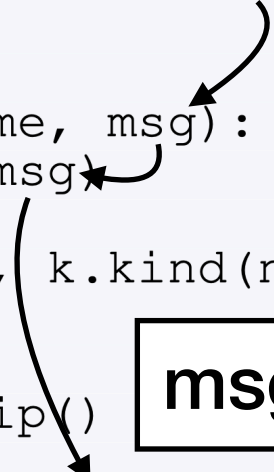
```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def ex
11     x =
12     path
13     retu
14
15 def pa
16     prin
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

정적분석 없이 무조건
LLM 호출

기존 도구들의 문제점

정적분석 제대로 활용 못해

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```



msg: str 추론

기존 도구들의 문제점

정적분석 제대로 활용 못해

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

Forward Inference로는
타입을 충분히 달아주지 못한다

msg: str 추론

기존 도구들의 문제점

함수 호출순서 마음대로

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

기존 도구들의 문제점

함수 호출순서 마음대로

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def ex
11     x =
12     path
13     retu
14
15 def pa
16     prin
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

마구잡이로 함수 골라
LLM에게 넘기기

기존 도구들의 문제점

함수 호출순서 마음대로

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

Callee들부터 추론

기존 도구들의 문제점

함수 호출순서 마음대로

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

Forward Inference로는
아무런 타입 정보도
전파가 불가능

Callee들부터 추론

기존 도구들의 문제점

컨텍스트 생성 대충

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```


기존 도구들의 문제점

컨텍스트 생성 대충

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

norm 함수 선택

기존 도구들의 문제점

컨텍스트 생성 대충

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

Caller Context 포함

기존 도구들의 문제점

컨텍스트 생성 대충

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name, p)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

Caller Context 포함

paths를 몰라 적절한 추론 불가능

기존 도구들의 문제점

정적분석 제대로 활용 못해

함수 호출순서 마음대로

컨텍스트 생성 대충

우리의 도구

정적분석 제대로 활용 **잘해**

함수 호출순서 **똑똑하게**

컨텍스트 생성 **깔끔**

우리의 도구

정적분석 제대로 활용 **잘해**

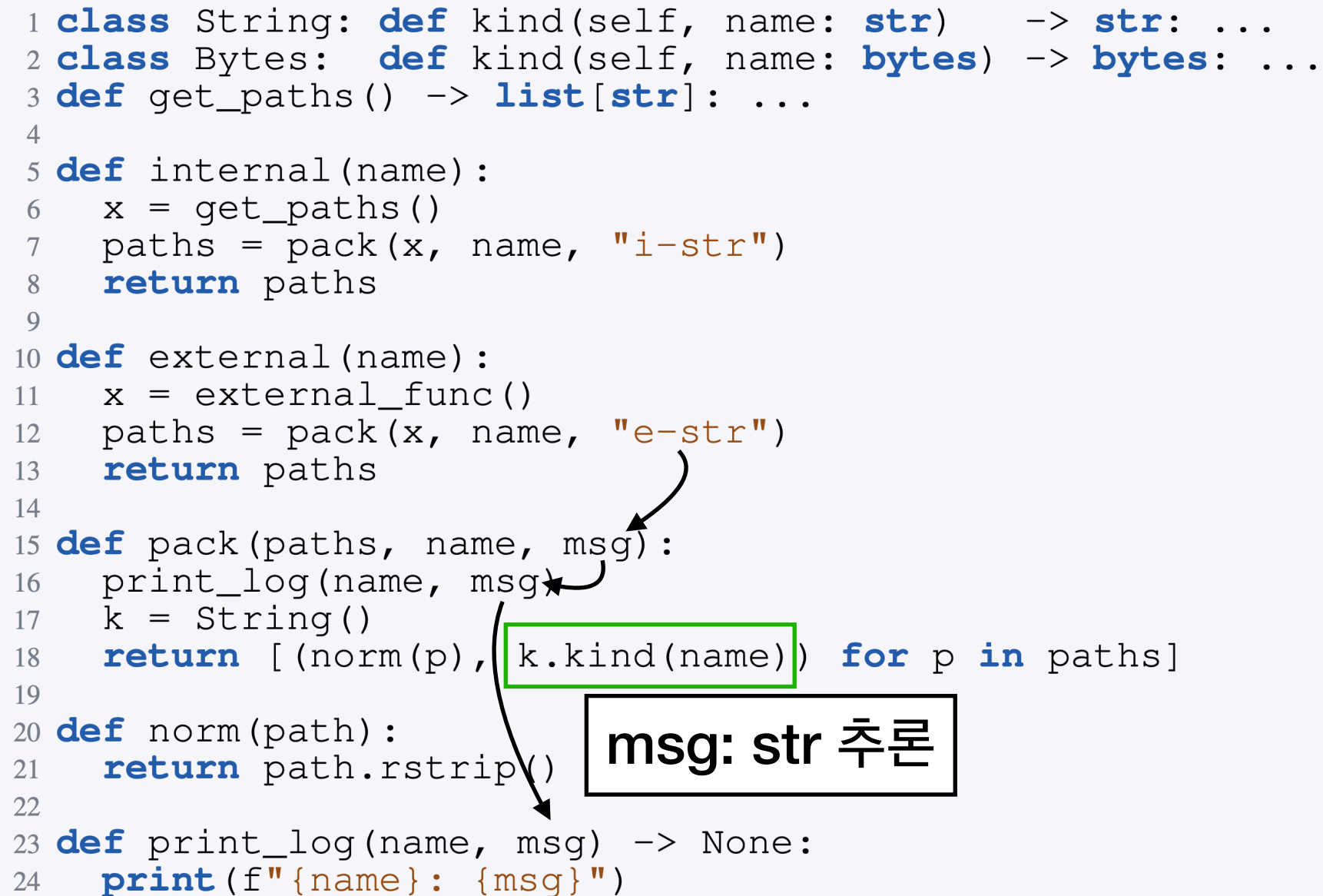
```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

msg: str 추론

우리의 도구

정적분석 제대로 활용 **잘해**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```



우리의 도구

정적분석 제대로 활용 **잘해**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name, msg) -> None:
24     print(f"{name}: {msg}")
```

name은 반드시
str 타입

msg: str 추론

우리의 도구

정적분석 제대로 활용 **잘해**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name, msg):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def print_log(name, msg) -> None:
21     print(f"{name}: {msg}")
```

name은 반드시 str 타입

name: str 추론

msg: str 추론

LLM 호출 없이도 함수 추론 성공!

우리의 도구

정적분석 제대로 활용 **잘해**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

우리의 도구

함수 호출순서 **똑똑하게**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

우리의 도구

함수 호출순서 **똑똑하게**

정적분석 효과를
최대로 끌어낼 수 있게

```
1 class String: def kind(self, name: str) -> str:
2 class Bytes: def kind(self, name: str) -> str:
3 def get_paths() -> list[str]:
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

우리의 도구

함수 호출순서 **똑똑하게**

```
1 class String: def kind(self, name: str) -> str:
2 class Bytes: def kind(self, name: str) -> str:
3 def get_paths() -> list[str]:
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

정적분석 효과를
최대로 끌어낼 수 있게

Caller/Callee가
가장 많은 함수 선택

우리의 도구

컨텍스트 생성 **깔끔**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

우리의 도구

컨텍스트 생성 **칼کم**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

Caller Context 포함

우리의 도구

컨텍스트 생성 **깔끔**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

Caller Context 포함

우리의 도구

컨텍스트 생성 **깔끔**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = external_func()
12     paths = pack(x, name, "e-str")
13     return paths
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

무한히 넣을 것인가?

Caller Context 포함

우리의 도구

컨텍스트 생성 **깔끔**

```
1 class String: def kind(self, name: str) -> str: ...
2 class Bytes: def kind(self, name: bytes) -> bytes: ...
3 def get_paths() -> list[str]: ...
4
5 def internal(name: str):
6     x = get_paths()
7     paths = pack(x, name, "i-str")
8     return paths
9
10 def external(name: str):
11     x = ext
12     paths =
13     return
14
15 def pack(paths, name: str, msg: str):
16     print_log(name, msg)
17     k = String()
18     return [(norm(p), k.kind(name)) for p in paths]
19
20 def norm(path):
21     return path.rstrip()
22
23 def print_log(name: str, msg: str) -> None:
24     print(f"{name}: {msg}")
```

Caller Context 포함

무한히 넣는 것인가?

x: list[str] (타입 요약정보)

우리의 도구

Backward 분석을 넣어
타입정보 최대한 모으기

정적분석 제대로 활용 **잘해**

함수 호출순서 **똑똑하게**

정적분석 효과를
최대로 끌어낼 수 있게

컨텍스트 생성 **깔끔**

타입 정보로
컨텍스트 최소화

결과

- 3k~96k까지 작고 큰 프로그램 12개 선정
- 실험엔 Gemma4-4b, Gpt-oss-120b, Glm-4.7 활용

System	Model	Accuracy	Time (hh:mm)	LLM Usage			
				Calls	Input (M)	Output (M)	Price (\$)
TIGER	CodeT5 (Fine-tuned)	49.83%	17:22	–	–	–	–
TYPET5	CodeT5 (Fine-tuned)	69.30%	01:53	–	–	–	–
TYPEGEN	Gpt-oss	46.51%	08:00	13,702	21.48	4.37	\$10.79
AIDER	Gemma4	72.04%	26:12	5,507	126.83	1.09	–
SALT	Gemma4	(↑6.6%) 76.76%	(↓32.6%) 17:40	(↓40.1%) 3,300	(↓79.0%) 26.63	(↓69.9%) 0.33	–
AIDER	Gpt-oss	82.41%	15:12	4,733	95.33	2.52	\$35.25
SALT	Gpt-oss	(↑2.0%) 84.06%	(↓6.0%) 14:17	(↓33.1%) 3,167	(↓76.1%) 22.78	(↓14.2%) 2.16	(↓72.8%) \$9.60
AIDER	Glm-4.7	84.13%	16:15	4,752	119.16	3.77	\$285.08
SALT	Glm-4.7	(↑2.0%) 85.82%	(↓4.4%) 15:32	(↓33.8%) 3,146	(↓80.9%) 22.77	(↓41.2%) 2.22	(↓78.5%) \$61.19

결과

- 3k~96k까지 작고 큰 프로그램 12개 선정
- 실험엔 Gemma4-4b, Gpt-oss-120b, Glm-4.7 활용

System	Model	Accuracy	기존 기술들보다 Code Assistant가 잘함				LM Usage		
							Input (M)	Output (M)	Price (\$)
TIGER	CodeT5 (Fine-tuned)	49.83%					—	—	—
TYPET5	CodeT5 (Fine-tuned)	69.30%	01:53	—	—	—	—	—	—
TYPEGEN	Gpt-oss	46.51%	08:00	13,702	21.48	4.37			\$10.79
AIDER	Gemma4	72.04%	26:12	5,507	126.83	1.09			—
SALT	Gemma4	(↑6.6%) 76.76%	(↓32.6%) 17:40	(↓40.1%) 3,300	(↓79.0%) 26.63	(↓69.9%) 0.33			—
AIDER	Gpt-oss	82.41%	15:12	4,733	95.33	2.52			\$35.25
SALT	Gpt-oss	(↑2.0%) 84.06%	(↓6.0%) 14:17	(↓33.1%) 3,167	(↓76.1%) 22.78	(↓14.2%) 2.16			(↓72.8%) \$9.60
AIDER	Glm-4.7	84.13%	16:15	4,752	119.16	3.77			\$285.08
SALT	Glm-4.7	(↑2.0%) 85.82%	(↓4.4%) 15:32	(↓33.8%) 3,146	(↓80.9%) 22.77	(↓41.2%) 2.22			(↓78.5%) \$61.19

결과

- 3k~96k까지 작고 큰 프로그램 12개 선정
- 실험엔 Gemma4-4b, Gpt-oss-120b, Glm-4.7 활용

System	Model	Accuracy	Time (hh:mm)	LLM Usage			
				Calls	Input (M)	Output (M)	Price (\$)
TIGER	CodeT5 (Fine-tuned)	49.83%	17:22	—	—	—	—
TYPET5	CodeT5 (Fine-tuned)	69.30%	01:53	—	—	—	—
TYPEGEN	Gpt-oss	46.51%	08:00	13,702	21.48	4.37	\$10.79
AIDER	Gemma4	72.04%	26:12	5,507	126.83	1.09	—
SALT	Gemma4	(↑6.6%) 76.76%	(↓32.6%) 17:40	(↓40.1%) 3,300	(↓79.0%) 26.63	(↓69.9%) 0.33	—
AIDER	Gpt-oss	82.41%	15:12	4,733	95.33	2.52	\$35.25
SALT	Gpt-oss	(↑2.0%) 84.06%	(↓6.0%) 14:17	(↓33.1%) 3,167	(↓76.1%) 22.78	(↓14.2%) 2.16	(↓72.8%) \$9.60
AIDER	Glm-4.7	84.13%	16:15	4,752	119.16	3.77	\$285.08
SALT	Glm-4.7	(↑2.0%) 85.82%	(↓4.4%) 15:32	(↓33.8%) 3,146	(↓80.9%) 22.77	(↓41.2%) 2.22	(↓78.5%) \$61.19

Aider 대비 Ours가 성능/시간/비용 측면에서 모두 우월

결과

- GPT-5.5에 대해서도 실험
- 비용 때문에 문제의 10%에 대해서만 확인해봄

Project	Sample Slots	Accuracy		Calls		Price (\$)	
		AIDER	SALT	AIDER	SALT	AIDER	SALT
black	325	89.23%	90.77%	119	93	21.43	5.38
fastapi	52	94.23%	94.23%	12	11	1.46	0.60
flake8	115	92.17%	93.04%	48	29	3.24	0.85
flask	160	88.12%	88.75%	63	44	7.91	1.72
poetry	160	98.12%	96.25%	90	57	10.25	1.96
pre-commit	130	96.15%	96.15%	52	34	4.53	0.70
pc-hooks	30	90.00%	96.67%	13	5	0.44	0.08
private-gpt	38	97.37%	100.00%	15	13	1.08	0.29
rich	82	100.00%	97.56%	34			
typer	133	90.23%	87.97%	29			
urllib3	138	94.93%	92.75%	61			
vnpy	243	92.18%	95.06%	104	70	9.41	2.31
Overall	1,606	92.71%	93.09%	640	448	73.42	18.49

Ours가 성능/비용 모두 우월

결론

- 파이썬 타입추론에 대하여
기존 도구들은 정적분석을 제대로 활용 못하고 있었음
- 정적분석을 제대로 활용하면
LLM 호출을 더 효과적/효율적으로 할 수 있음
- 성능은 오르면서도
비용은 약 70% 절감 가능 하였음